

**LAPORAN PRAKTIKUM SISTEM OPERASI
MANAJEMEN PROSES DAN CHAT IPC**



**Dosen Pengampu :
Triawan Adi Cahyanto M.Kom**

**Disusun Oleh :
Subhan Maulana Andrianto 2410651014**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER**

2025

MANAJEMEN_PROSES

Manajemen proses adalah fungsi dari sistem operasi yang mengatur eksekusi program-program (proses) yang berjalan di komputer, mulai dari pembuatan, penjadwalan, sinkronisasi, komunikasi antar proses, hingga penghentian proses. Lebih spesifik: proses adalah “suatu program yang sedang dieksekusi” bersama dengan aktivitasnya dan sumber daya yang dibutuhkan (memori, CPU, I/O) — manajemen proses bertugas mengatur sumber daya tersebut agar proses berjalan dengan baik.

Tujuan utama manajemen proses adalah agar sistem komputer dapat menjalankan banyak proses secara efisien, terorganisir, dan adil. Berikut beberapa tujuannya, antara lain :

- Memastikan penggunaan CPU, memori, I/O secara optimal sehingga tidak terjadi pemborosan atau idle yang besar.
- Memberikan respons yang baik dan waktu eksekusi yang wajar untuk proses-proses yang berjalan (penjadwalan yang tepat).
- Mengatur komunikasi dan sinkronisasi antar proses agar proses yang saling terkait tidak saling mengganggu dan deadlock atau kondisi race bisa diminimalkan.
- Menyediakan lingkungan di mana proses-baru dapat dibuat dan proses-selesai dapat dihapus dengan rapi sehingga sumber daya bisa dikembalikan dan digunakan oleh proses lain.

INPUT:

```
[1/1] manajemen_proses.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

double sekarang_detik() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

void tugas_sort(int n) {
    int *a = malloc(sizeof(int) * n);
    if (!a) { perror("malloc"); exit(1); }
    for(int i = 0; i < n; i++) a[i] = rand() % 100000;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n-1-i; j++){
            if(a[j] > a[j+1]){
                int t = a[j];
                a[j] = a[j+1];
                a[j+1] = t;
            }
        }
    }
    free(a);
}
```

```

}

void tugas_cari(int n, int kunci) {
    int *a = malloc(sizeof(int) * n);
    if (!a) { perror("malloc"); exit(1); }
    for(int i = 0; i < n; i++) a[i] = rand() % 100000;
    int ditemukan = 0;
    for(int i = 0; i < n; i++){
        if(a[i] == kunci) { ditemukan = 1; break; }
    }
    free(a);
}

long tugas_hitung(int n) {
    long jumlah = 0;
    for(int i = 2; i < n; i++){
        int prima = 1;
        for(int j = 2; j*j <= i; j++){
            if(i % j == 0) { prima = 0; break; }
        }
        if(prima) jumlah += i;
    }
    return jumlah;
}

int main() {
    pid_t pid;

```

```

    int status;
    double mulai, selesai;

    srand(time(NULL));

    for(int tugas = 1; tugas <= 3; tugas++){
        mulai = sekarang_detik();
        pid = fork();
        if(pid < 0) {
            perror("fork");
            exit(1);
        }
        if(pid == 0) {
            pid_t pid_anak = getpid();
            printf("Anak PID %d mulai. PID = %d\n", tugas, pid_anak);
            if(tugas == 1) {
                tugas_sort(200000);
                printf("Anak PID %d: tugas pengurutan selesai\n", pid_anak);
            }
            else if(tugas == 2) {
                tugas_cari(500000, rand()%100000);
                printf("Anak PID %d: tugas pencarian selesai\n", pid_anak);
            }
            else if(tugas == 3) {
                long res = tugas_hitung(200000);
                printf("Anak PID %d: tugas perhitungan selesai. Jumlah prima = %d\n", pid_anak, res);
            }
        }
    }

```

```

        selesai = sekarang_detik();
        double waktu = selesai - mulai;
        printf("Anak PID %d selesai. Waktu eksekusi = %.6f detik. Status keluar = 0\n",
            pid_anak, waktu);
        exit(0);
    }
    else {
        printf("Induk: membuat anak untuk tugas %d dengan PID = %d\n", tugas, pid);
    }
}

while((pid = wait(&status)) > 0) {
    if(WIFEXITED(status)) {
        int kode = WEXITSTATUS(status);
        printf("Induk: anak PID %d selesai. Kode keluar = %d\n", pid, kode);
    } else if(WIFSIGNALED(status)) {
        int sig = WTERMSIG(status);
        printf("Induk: anak PID %d dihentikan oleh sinyal %d\n", pid, sig);
    } else {
        printf("Induk: anak PID %d berakhir tidak normal\n", pid);
    }
}

printf("Induk: semua anak selesai.\n");
return 0;
}

```

OUTPUT :

```
root@DESKTOP-J9C60BQ:~# gcc -o manajemen_proses manajemen_proses.c
root@DESKTOP-J9C60BQ:~# ./manajemen_proses
Induk: membuat anak untuk tugas 1 dengan PID = 564
Anak (tugas 1) mulai. PID = 564
Induk: membuat anak untuk tugas 2 dengan PID = 565
Anak (tugas 2) mulai. PID = 565
Induk: membuat anak untuk tugas 3 dengan PID = 566
Anak (tugas 3) mulai. PID = 566
Anak PID 565: tugas pencarian selesai
Anak PID 565 selesai. Waktu eksekusi = 0.036298 detik. Status keluar = 0
Induk: anak PID 565 selesai. Kode keluar = 0
Anak PID 566: tugas perhitungan selesai. Jumlah prima = 1709600813
Anak PID 566 selesai. Waktu eksekusi = 0.093869 detik. Status keluar = 0
Induk: anak PID 566 selesai. Kode keluar = 0
Anak PID 564: tugas pengurutan selesai
Anak PID 564 selesai. Waktu eksekusi = 3.027273 detik. Status keluar = 0
Induk: anak PID 564 selesai. Kode keluar = 0
Induk: semua anak selesai.
```

ANALISIS :

- Setiap proses anak melakukan tugas yang berbeda (sorting, searching, perhitungan), sehingga waktu eksekusi akan sangat bervariasi antar anak — tugas sorting mungkin memakan waktu cukup lama dibanding tugas searching atau perhitungan ringan.
- Waktu eksekusi yang tercetak (misalnya “Waktu eksekusi = X detik”) menunjukkan efisiensi tugas, jika waktu terlalu besar maka bisa menunjukkan bahwa ukuran tugas terlalu besar atau bahwa algoritma yang digunakan kurang optimal (misalnya bubble sort).
- Koordinasi lewat proses induk (wait()) memastikan bahwa resource (CPU) dibagi dan proses anak selesai sebelum induk mencatat hasil akhir, ini menandakan manajemen proses berjalan dengan baik.
- Namun, karena setiap tugas berjalan sekuensial di dalam child masing-masing tanpa paralel aktif antar anak yang saling banyak berinteraksi, maka potensi maksimal paralelisme belum sepenuhnya dimanfaatkan, artinya jika sistem mempunyai banyak CPU/inti, bisa menjalankan lebih banyak anak atau tugas yang lebih berat untuk menguji skala.

CHAT_IPC

IPC adalah mekanisme yang memungkinkan dua atau lebih proses yang berjalan dalam sistem operasi untuk “berbicara”, berbagi data, atau menyinkronkan aktivitas para pengguna, yang bertujuan untuk :

1. Memfasilitasi komunikasi antar proses,
2. Menyinkronkan akses ke resource bersama,
3. Efisiensi dan performa,
4. Membangun aplikasi modular / proses-terpisah,
5. Demonstrasi dan eksperimen konsep sinkronisasi & komunikasi proses.

INPUT :

```
GNU nano 7.2 chat_ipc.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <semaphore.h>

#define N_MAX_PENGGUNA 10
#define PENGHITUNG 100
#define MEM_NAMA "/mem_chat"
#define SEM_PENGIRIM "/sem_pengirim"
#define SEM_PENERIMA "/sem_penerima"

typedef struct {
    int indeks_terakhir;
    char pesan[PENGHITUNG][256];
    pid_t pengirim[PENGHITUNG];
} StrukturChat;

int main(int argc, char *argv[]) {
    int fd;
    StrukturChat *chat;
    sem_t *semKirim, *semTerima;
    pid_t pid = getpid();

    char input[256];
    int pilihan;

    fd = shm_open(MEM_NAMA, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, sizeof(StrukturChat));
    chat = mmap(NULL, sizeof(StrukturChat), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    semKirim = sem_open(SEM_PENGIRIM, O_CREAT, 0666, 1);
    semTerima = sem_open(SEM_PENERIMA, O_CREAT, 0666, 0);

    if (chat->indeks_terakhir < 0 || chat->indeks_terakhir >= PENGHITUNG) {
        chat->indeks_terakhir = 0;
    }

    printf("Aplikasi Chat - PID %d\n", pid);

    while (1) {
        printf("\nMenu\n1. Kirim pesan\n2. Lihat history\n3. Exit\nPilihan: ");
        scanf("%d", &pilihan);
        getchar();

        if (pilihan == 1) {
            printf("Tulis pesan: ");
            fgets(input, sizeof(input), stdin);
            input[strcspn(input, "\n")] = '\0';

            sem_wait(semKirim);
        }
    }
}
```

```

        int idx = chat->indeks_terakhir % PENGHITUNG;
        chat->pengirim[idx] = pid;
        strncpy(chat->pesan[idx], input, sizeof(chat->pesan[idx])-1);
        chat->pesan[idx][sizeof(chat->pesan[idx])-1] = '\0';
        chat->indeks_terakhir++;
        sem_post(semTerima);

        printf("Pesan terkirim.\n");
    }
    else if (pilihan == 2) {
        sem_wait(semTerima);
        int batas = chat->indeks_terakhir;
        printf("History pesan (terbaru ke terlama):\n");
        for (int i = 0; i < batas; i++) {
            int idx = (chat->indeks_terakhir - 1 - i + PENGHITUNG) % PENGHITUNG;
            printf("[%d] PID %d: %s\n", idx, chat->pengirim[idx], chat->pesan[idx]);
        }
        sem_post(semKirim);
    }
    else if (pilihan == 3) {
        break;
    }
    else {
        printf("Pilihan tidak valid.\n");
    }
}

```

```

munmap(chat, sizeof(StrukturChat));
shm_unlink(MEM_NAMA);
sem_close(semKirim);
sem_close(semTerima);
sem_unlink(SEM_PENGIRIM);
sem_unlink(SEM_PENERIMA);

printf("Keluar aplikasi chat. PID %d\n", pid);
return 0;
}

```

OUTPUT :

```
root@DESKTOP-J9C60BQ:~# gcc chat_ipc.c -o chat_ipc -lrt -pthread
^[[Aroot@DESKTOP-J9C60BQ:~/chat_ipc
Aplikasi Chat - PID 531

Menu
1. Kirim pesan
2. Lihat history
3. Exit
Pilihan: 1
Tulis pesan: Halo, apa yang sedang anda lakukan ?
Pesan terkirim.

Menu
1. Kirim pesan
2. Lihat history
3. Exit
Pilihan: 1
Tulis pesan: halo juga, saya sedang mengerjakan tugas
Pesan terkirim.

Menu
1. Kirim pesan
2. Lihat history
3. Exit
Pilihan: 2
History pesan (terbaru ke terlama):
[1] PID 531: halo juga, saya sedang mengerjakan tugas
[0] PID 531: Halo, apa yang sedang anda lakukan ?
```

ANALISIS PERFORMA :

- Karena menggunakan mekanisme Shared Memory bersama dengan sinkronisasi semaphore, komunikasi antar proses dapat berjalan dengan **latensi rendah** dan throughput relatif tinggi dibanding mekanisme IPC berbasis pesan yang melibatkan lebih banyak penyalinan data dan panggilan sistem.
- Secara keseluruhan untuk aplikasi chat ringan (pesan teks pendek antar beberapa proses pada satu mesin), performa akan sangat memadai: respons cepat, hampir real-time.

KESIMPULAN

- **Manajemen Proses**

Praktikum ini menekankan aspek-kunci dari sistem operasi dalam mengelola proses: penciptaan (fork), eksekusi tugas berbeda, koordinasi antara proses induk dan anak, pengukuran waktu eksekusi, dan terminasi proses. Tujuannya adalah untuk memahami bagaimana sistem operasi memastikan proses berjalan secara efisien, sumber daya (CPU, memori) digunakan secara optimal, dan tidak terjadi konflik antar proses.

- **Chat IPC (Inter-Process Communication)**

Melalui aplikasi chat sederhana dengan shared memory/semaphore atau message queue, praktikum ini memperlihatkan bagaimana proses-berbeda dapat saling berkomunikasi dan berbagi data. Fokusnya pada bagaimana mengatur media bersama (shared memory atau message queue), serta menerapkan sinkronisasi agar tidak terjadi race condition atau konflik akses. Tujuannya adalah untuk memahami mekanisme IPC dan bagaimana proses bisa kolaborasi dalam satu sistem.

Secara keseluruhan dari dua metode diatas ialah, praktikum tersebut memperkuat pemahaman praktis bahwa manajemen proses dan komunikasi antar proses (IPC) adalah dua pilar penting dalam desain sistem operasi dan aplikasi terdistribusi, yang saling berkaitan dan saling membutuhkan.

PRAKTIKUM

- Percobaan 1.1 - Membuat proses sederhana

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
```

```
int main() {
    pid_t pid;
```

```
    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
    printf("PID parent: %d\n", getppid());
```

```
    return 0;
}
```

```
root@DESKTOP-J9C60BQ:~# gcc -o process_basic process_basic.c
```

```
root@DESKTOP-J9C60BQ:~# ./process_basic
```

```
Program dimulai
```

```
PID proses ini: 1146
```

```
PID parent: 320
```

```
root@DESKTOP-J9C60BQ:~# |
```

- **Percobaan 1.2 – Fork: Parents dan Child Process**

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());

    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    }

    else if (pid == 0) {
        //Child process
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }

    else {
```

```
root@DESKTOP-J9C60BQ:~# gcc -o fork_demo fork_demo.c
root@DESKTOP-J9C60BQ:~# ./fork_demo
Sebelum fork()
PID: 1195

PARENT PROCESS
PID saya: 1195
PID child saya: 1196
Proses 1195 selesai

CHILD PROCESS
PID saya: 1196
PID child saya: 0
Proses 1196 selesai

root@DESKTOP-J9C60BQ:~# |
```

Percobaan 2.1-pipes: One Way Communication